Informàtica 1

Semester 2026-27 Q1

Airline Management

Project statement

Version 2

The application must allow the assignment of scheduled flights for a given day to the aircraft available that day. For example, on a given day, the airline has a list of scheduled flights and has aircraft parked at airports around the world at the start of the day. To assign a flight to an aircraft, the aircraft's capacity must be sufficient, and the aircraft must be at the departure airport at the scheduled departure time. If that flight goes from El Prat to Heathrow, then that same aircraft could be assigned to a second flight departing from Heathrow at a later time. This assignment will be the main focus of version 2, together with new functionalities to build graphics showing the routes assigned to each aircraft for the day.

The development of version is much less guided than the text for version 1. Read carefully all the requirements and clues that will be described in the following paragraphs and consider the distribution of tasks between team members. You will have to modify functions from version 1 and include new functions (and maybe new classes). The test section in every class file must include the code to test all functions in that file, including the new ones.

To manage flight assignments to aircraft, it will be necessary to add to the Flight class some new attribute that indicates whether that flight already has an aircraft assigned and, if so, which aircraft.

It will also be necessary to implement a function that assigns aircrafts to flights. This function will have to iterate through the flight list and, for each flight, find an aircraft that can handle it. Naturally, it will have to be an aircraft with sufficient capacity, available during the scheduled flight time, and that is in the departure airport at the time of departure.

Note that in order to implement the assignment for every day we need to know in which airport is every aircraft at the beginning of the day. You will need to include this information in the definition of class Aircraft and include the corresponding data in the text file for aircrafts.

Since the aircraft may end the day at a different airport than the one it was at the beginning of the day, it will be necessary to add the functionality required to generate, at the end of the day, a new aircraft file with the correct locations of each one at the end of the day, which will be used for assignments of the flights scheduled for the following day.

Two versions of the assignment algorithm must be considered. The first version (algorithm A) must assume that each aircraft will make only one flight that day (or none). To do this, the algorithm must iterate through the list of scheduled flights and, for each one, search for an aircraft to which that flight can be assigned; that is, an aircraft with sufficient capacity that is at the start of the day at the airport from which the flight departs.

For the second version of the assignment algorithm (algorithm B), we will assume that each aircraft can make up to two flights per day. To assign two flights to the same aircraft, the aircraft's capacity must be sufficient for each flight, the arrival airport of the first flight must be the same as the departure airport of the second, and there must be at least two hours between the arrival of the first flight and the departure of the second.

Obviously, it's possible that not all flights scheduled for the day can be assigned. The application must display to the user both the list of assigned flights and the list of unassigned flights, using either of the two assignment algorithms.

Two types of graphs must be included in version 2. First, the graph shown in version 1 must be modified to also display flights in the air during each one-hour period as a result of applying each of the two assignment algorithms. This will allow for a visual assessment of the effectiveness of the two algorithms. Second, a geolocated map must be displayed showing the planned routes for each aircraft after the assignment process. Ideally, the user should be able to select an aircraft, and the application will display the routes for that aircraft. To perform this function, the geographic coordinates (latitude and longitude) of each airport must be known. A file containing this information for a representative set of airports can be found in Atenea.

Finally, you must extend the user interface with the elements that should allow the user to use the new functionalities.

Step description

Step 1 – Aircraft initial location

Extend the Aircraft class to include the airport where each aircraft is located at the beginning of the day. Modify the functions that load and save aircraft information so that this additional attribute is included in the corresponding text files. Verify that aircraft locations are correctly loaded, displayed and saved.

Step 2 – Airport information management

Incorporate airport information into the application. Each airport must have an ICAO code and its geographical coordinates (latitude and longitude). Implement the functionalities required to load, save, display, add and remove airports. You can use the airport data file provided in Atenea to test your implementation. Decide how this information should be represented and integrated with the existing application.

Step 3 – Flight assignment information

Extend the Flight class to keep track of whether a flight has been assigned to an aircraft and, if so, which aircraft has been assigned. Make sure the application can distinguish between assigned and unassigned flights and display this information to the user. Consider how to initialize or reset assignments before executing an assignment algorithm.

Step 4 – Assignment algorithm A

Implement the first aircraft assignment algorithm. This algorithm must process the scheduled flights and try to assign an available aircraft to each flight, considering the following conditions:

- Each aircraft can perform at most one flight per day.
- The aircraft must have sufficient capacity for the passengers.
- The aircraft must be located at the departure airport at the beginning of the day.
- An aircraft already assigned to another flight cannot be assigned again.

The application must display the list of assigned flights, indicating the corresponding aircraft, and the list of flights that could not be assigned. Test the algorithm using different combinations of aircraft and scheduled flights.

Step 5 – Assignment algorithm B

Implement a second assignment algorithm that allows each aircraft to perform up to two flights per day. In addition to the capacity restrictions, two flights can be assigned to the same aircraft only if:

- - The arrival airport of the first flight matches the departure airport of the second.
- - There are at least two hours between the arrival of the first flight and the departure of the second.
- - The aircraft does not exceed the maximum of two flights per day.

The algorithm must identify which flights can be assigned and which cannot. Make sure that the results of algorithms A and B can be obtained independently, without the assignments made by one algorithm interfering with the other.

Step 6 – End-of-day aircraft locations

After executing an assignment algorithm, determine the airport where each aircraft will be located at the end of the day. An aircraft that has performed one or two flights will end the day at the arrival airport of its last assigned flight. Aircraft that have not performed any flight will remain at their initial airport. Implement the functionality required to generate a new aircraft data file containing these updated locations. This file must be compatible with the aircraft loading functionality and should be usable as the initial fleet information for the following day.

Step 7 – Graphical representation of results

Extend the graphical capabilities of the application with two new functionalities.

- Flight assignment comparison: Modify the graph developed in Version 1 to compare the number of scheduled flights with the number of flights assigned by algorithms A and B during each one-hour period. The graph must allow the user to visually compare the results obtained with the two algorithms.
- Aircraft routes: Implement a graphical representation of the routes assigned to the aircraft, using the geographical coordinates of the airports. The user should be able to select an aircraft and visualize its assigned route or routes on a map.

Step 8 – Graphical user interface and integration

Extend the existing tkinter interface to incorporate all the new functionalities. The user must be able to manage airport information, execute either assignment algorithm, inspect assigned and unassigned flights, generate updated aircraft files and display the new graphs. Verify that the new functionalities work correctly together with those developed in Version 1.

Include appropriate tests for the new functions and update the project documentation.

Additional quality criteria for version 2

Correct

- The test section of every class file allows to test all the functions defined in the file.
- Through the graphical interface the user can:
  - o Load, show, add, remove and save airport information from/to text file
  - o Assign aircrafts to flight using algorithm A and show result to the user (both assigned and unassigned flights)
  - o Assign aircrafts to flight using algorithm B and show result to the user (both assigned and unassigned flights)
  - o Generate a new version of the aircraft file including the airport where the aircraft will be at the end of the day
  - o Show the plot that compares flights on air for each algorithm
  - o Plot routes for every aircraft